5bit — LEXICON & ENCODING SPECIFICATION

> Canonical reference for the 5bit fabric: the 32-token lexicon, the five contexts, the context-switching stack, and the encode/decode/pack rules.
>
> This document is the **contract**. Every binding (Python, TypeScript, C, Swift) MUST produce byte-identical output for the same input. Conformance is verified against fixed vectors; if a binding disagrees with this spec, the binding is wrong.
>
> Status legend: **[VERIFIED]** = confirmed against the reference implementation by execution. **[DESIGN]** = intended behavior / convention. **[NOTE]** = honest engineering caveat.

---

1. Core invariants

1. **Fixed width.** Every token is exactly **5 bits**. There are therefore **32 possible tokens** (`00000` – `11111`).
2. **Context-relative meaning.** 28 of the 32 codes (`00000` – `11011`) are *context-dependent*: the same 5 bits mean different things in different contexts. The remaining 4 codes (`11100` – `11111`) are **control tokens** with identical meaning in every context.
3. **Deterministic.** Encoding is a pure function of the input. No timestamps, no randomness, no platform-dependent behavior. Same input → same tokens → same bytes, on every language and machine. **[VERIFIED: Python ≡ C, 25/25 conformance vectors byte-identical.]**
4. **Big-endian bit packing.** Tokens are serialized most-significant-bit first and zero-padded at the **tail** to the next byte boundary.
5. **Append-only.** Records are never overwritten in place; new versions are appended. This is what gives MVCC-style snapshot semantics structurally.

---

2. The 32-token lexicon (five contexts)

28 mappable slots per context (`00000` – `11011`) + 4 shared control tokens (`11100` – `11111`).

Binary	Dec	NUM	WORD	SPECIAL	SPECIAL2	SPECIAL3
00000	0	0	A	a	!	AUTH
00001	1	1	B	b	"	GRANT_R
00010	2	2	C	c	#	GRANT_W
00011	3	3	D	d	$	REVOKE
00100	4	4	E	e	%	ENCRYPT
00101	5	5	F	f	&	LABEL
00110	6	6	G	g	'	—
00111	7	7	H	h	(	—
01000	8	8	I	i	)	—
01001	9	9	J	j	*	—
01010	10	+	K	k	+	—
01011	11	-	L	l	,	—
01100	12	*	M	m	/	—
01101	13	/	N	n	:	—
01110	14	=	O	o	;	—
01111	15	(	P	p	<	—
10000	16	)	Q	q	=	—
10001	17	-1	R	r	>	—
10010	18	-2	S	s	?	—
10011	19	-3	T	t	[	—
10100	20	-4	U	u	\	—
10101	21	-5	V	v	]	—
10110	22	-6	W	w	^	—

10111	23	-7	X	x	_	—
11000	24	-8	Y	y	`	—
11001	25	-9	Z	z	{	—
11010	26	^	␣	@	|	—
11011	27	S *	.	-	}	—
11100	28	**RECORD** — control, all contexts				
11101	29	**CHECKSUM** — control, all contexts				
11110	30	**END** — control, all contexts				
11111	31	**START** — control, all contexts				

* NUM slot 27 (`S`) is the **scale marker** used to encode decimals as `mantissa × 10-ˢ` (e.g. `0.123456 = 123456 × 10-⁶`). Slots 10–16 in NUM are arithmetic operators; 17–25 are the **negative digits** `-1…-9` ; slot 26 (`^`) is power.

Control tokens (constant across every context)

Token	Dec	Role
START	31	Push one context deeper (NUM→WORD→SPECIAL→SPECIAL2→SPECIAL3).
END	30	Pop one context shallower. At WORD, pop finalizes back to NUM.
RECORD	28	Terminate the current logical tuple; finalize and reset to NUM. Boundary for geometric (record-vs-record) queries.
CHECKSUM	29	Emit an integrity marker.

---

3. Context-switching stack

The parser is a 5-level stack machine. It **starts in NUM**.

Token	From state	Action
START	NUM	→ WORD
START	WORD	→ SPECIAL
START	SPECIAL	→ SPECIAL2
START	SPECIAL2	→ SPECIAL3 (control-command context)
END	SPECIAL3	→ SPECIAL2
END	SPECIAL2	→ SPECIAL
END	SPECIAL	→ WORD
END	WORD	→ NUM (finalizes the word)
RECORD	any	finalize, emit record boundary, → NUM
CHECKSUM	any	emit integrity marker (state unchanged)

Depth convention used by the encoder: `0=NUM`, `1=WORD` -ish content layer, escalating for SPECIAL/SPECIAL2. "Pop to depth *d*" means emit `END` until the current depth equals *d*.

---

4. Encoding — integers [VERIFIED]

```
encode_integer(value):
    if value == 0:
        return [D0, END]                    # [0, 30]
    neg    = value < 0
    digits = decimal digits of |value|, most-significant first
    out    = []
    for d in digits:
        if d == 0:        out.append(D0)          # token 0, regardless of sign
        elif neg:         out.append(N{d})        # token 16 + d   (N1=17 … N9=25
        else:             out.append(D{d})        # token d        (D1=1  … D9=9)
    out.append(END)                                # token 30
    return out
```

Rules that matter:

- A `0` digit is **always** `D0` (token 0), even inside a negative number. Sign is carried by the

non-zero digits.

- Leading zeros are preserved as literal `D0` tokens (so `"007"` ≠ integer `7` when decoded as text — see §7).

---

5. Encoding — words / strings [VERIFIED]

```
encode_word(text):
    out   = [START]            # enter WORD
    depth = 0
    for ch in text:
        if ch is digit:
            pop_to(0); emit END; emit D{ch}; emit START   # dip to NUM for the di
        elif ch in WORD  (A-Z, space, '.'):
            pop_to(0); emit WORD_token(ch)
        elif ch in SPECIAL (a-z, '@', '-'):
            ensure depth == 1 (emit START or END as needed); emit SPECIAL_token(c
        elif ch in SPECIAL2 (punctuation):
            ensure depth == 2; emit SPECIAL2_token(ch)
        else:
            uppercase-fallback into WORD
    pop_to(0); emit END        # final WORD→NUM
    return out
```

Char → slot within a context is positional: `A→0 … Z→25`, `space→26`, `.→27` (WORD); `a→0 … z→25`, `@→26`, `-→27` (SPECIAL); punctuation in the fixed SPECIAL2 order `! " # $ % & ' ( ) * + , / : ; < = > ? [\ ] ^ _ \ {|}`` ` ``.

---

6. Bit packing [VERIFIED]

```
pack(tokens):
    bitstream = concatenation of each token as 5 bits, MSB-first
    total     = 5 * len(tokens)
    pad       = (8 - (total % 8)) % 8
    append `pad` zero bits to the TAIL
    return bytes (MSB-first), and pad

unpack(bytes, pad):
    total = len(bytes)*8 - pad
    read total bits MSB-first, slice into 5-bit groups → tokens
```

Reference vectors **[VERIFIED]** (`hex / pad`):

Input	Tokens	Bytes	Pad
int `0`	`D0 END`	`0780`	6
int `1`	`D1 END`	`0f80`	6
int `-1`	`N1 END`	`8f80`	6
int `42`	`D4 D2 END`	`20bc`	1
int `-42`	`N4 N2 END`	`a4bc`	1

---

7. Records & labels

Record boundary

A **RECORD** token terminates a logical tuple. Everything between two RECORD tokens is one record. Record *N* is addressable by position (alloc table or fixed stride), so primary-key-by-id lookup needs **no secondary index** — the address *is* the key.

Labels (self-describing schema)

`encode_label(position, name)` emits, in SPECIAL3:

```
START START START START   LABEL   D{position…} END END END END   START name… END
```

i.e. the LABEL command, the field position as a NUM, then the field name as a word. Labels move the schema **into the data**: a reader reconstructs `{name: value}` with no external catalog.

Canonical layout: INTERLEAVED [VERIFIED + DESIGN DECISION]

```
LABEL 0 "age"  <value0>   LABEL 1 "name"  <value1>   … RECORD
```

Each label is immediately followed by its value. `data_pos` is set by the label's explicit position; all tokens until the next LABEL belong to that bucket.

- **[NOTE]** *Labels-first* layout (all labels, then all values) is **not** canonical and must not be emitted: the reader cannot detect where the header ends, so values collapse into the

last label's bucket. Use interleaved only.

- **[NOTE]** `reconstruct_by_labels` reconstructs each field by joining all tokens in its bucket; this round-trips mixed values losslessly (e.g. `"id007"`, `"user 42"`, `"A1B2"`) **[VERIFIED]** when records are interleaved.

Interleaved binding rule [DESIGN]

In the canonical layout, every `LABEL` opens exactly one field, and that field stays open until the next control event closes it: the following `LABEL` (which opens the next field), a `RECORD` (which ends the tuple), or end of stream. The tokens between a label's metadata (its position marker and name) and the next `LABEL` / `RECORD` are that field's value, joined in order. This is precisely why interleaved is unambiguous and labels-first is not — the reader always knows a value has *begun* (a `LABEL` just closed) and when it *ends* (the next `LABEL` / `RECORD`), with no header boundary to infer. A direct consequence is that a well-formed stream **never places two `LABEL` commands back to back**: a `LABEL` immediately followed by another `LABEL` describes a field with no value, which is malformed — the SPECIAL3 equivalent of writing `TABLE TABLE` in SQL. A correct encoder cannot emit it, so the contract lives in the writer and the default parser does not pay to re-check it on the hot path.

Optional LABEL validation (opt-in) [DESIGN]

Because the rule is structural, it can be enforced cheaply *when you choose to* — e.g. when ingesting an untrusted or hand-assembled stream — with a small guard (~10 lines) that walks the SPECIAL3 commands and raises on a violation: two `LABEL`s with no intervening value, a `LABEL` missing its position/name, or a position marker with no name. This is a deliberate **double choice**: the trusted path stays allocation-free and check-free (write it right, or it's garbage-in-garbage-out, the C/C++ contract model), while callers who want a hard failure instead of silent collapse switch the guard on. It is *not* a tax on the default parser. Illustrative guard (adapt the metadata skip to your parsed token shape):

```
def validate_interleaved_labels(parsed):
    """Raise on malformed SPECIAL3 LABEL structure. Opt-in; not run on the hot pa
    is_label = lambda t: isinstance(t, dict) and t.get("cmd") == "LABEL"
    i, n = 0, len(parsed)
    while i < n:
        if is_label(parsed[i]):
            if i + 2 >= n:                      # need command + name + position
                raise ValueError("LABEL missing name/position")
            i += 3                              # skip the label's own metadata
            value_tokens = 0
            while i < n and not is_label(parsed[i]):  # consume until next field/
```

```
            value_tokens += 1
            i += 1
        if value_tokens == 0:                # LABEL immediately followed by L
            raise ValueError("malformed: LABEL with no value (LABEL LABEL)")
    else:
        i += 1
```

---

8. Decoding / reconstruction [VERIFIED]

1. `unpack(bytes, pad)` → tokens.
2. Run the §3 stack machine, accumulating per context:
   - NUM: collect signed digits; on START/END/RECORD finalize to an integer (`value = Σ digitᵢ · 10^(n-1-i)`).
   - WORD/SPECIAL/SPECIAL2: collect chars; on context change finalize to a string fragment via the inverse char map.
3. Fragments at a labeled position are concatenated in order → the field value. Digit text is preserved (e.g. leading zeros survive), so reconstruction is lossless.

---

9. Durability & concurrency

- **WAL.** Each entry = header + payload + **SHA-256(entry)**, `fsync` 'd, append-only. Corruption is detected on replay and located by sequence number
  - offset. **[VERIFIED: crash recovery via SIGKILL replays cleanly.]**
  - **[NOTE]** This is a **per-entry checksum**, not a value-chained ledger: each entry's hash incorporates the *offset* of the previous entry, not its *hash value*. It detects accidental corruption and is fully traceable, but is not tamper-evident against a deliberate edit that also rewrites that entry's own hash. To upgrade to a true chain, fold `prev_entry.sha256` into the hashed content.
- **CAS.** `write_if(id, tokens, expected_offset, expected_bitlen)` prevents lost updates under concurrent writers. **[VERIFIED: 12 processes, one row, zero lost updates.]**
- **Group commit.** Buffer writes, one `fsync` per batch. **[VERIFIED: matched-fsync durable write beats SQLite and Postgres on the embedded single-client path.]**

- **[NOTE]** Concurrency is currently serialized by a per-grid `flock`. Correct under contention, but parallel-writer throughput trails MVCC engines until per-record/sharded locking (or append-only versioning) is wired.

---

10. Access control [VERIFIED]

- `RLSEngine` extends the grid and overrides read/write/delete to require a `user_id`; the owner is stored at position 0 and checked on every access, raising `PermissionDenied` on mismatch. There is no raw grid handle to bypass. **[VERIFIED: cross-user read/write blocked; admin context bypasses.]**
- Per-user **AES-CTR** encryption keyed off a password is available.
- **[NOTE]** The SPECIAL3 `AUTH` / `GRANT_*` / `REVOKE` / `ENCRYPT` tokens are *representations* in the token stream. Enforcement is performed by `RLSEngine` (owner check + method-level guards), not by the tokens alone.

---

11. Determinism & conformance

- A binding is conformant iff, for every vector in the conformance suite, its `pack(encode(input))` bytes and pad equal the reference. **[VERIFIED: Python ≡ C across 25 vectors; Python ≡ TS across 53 vectors.]**
- New bindings MUST run the conformance suite as their first test. Bit-packing (endianness, tail padding, shift masking) is the most common source of divergence — use fixed-width unsigned types and mask every shift (`(x >> n) & 0x1F`).

---

12. Vector fingerprinting (embeddings) — 5bit has legs on vector

This is the part that makes 5bit a vector store, not just a record store, and it falls straight out of the lexicon with **nothing bolted on**.

12.1 An embedding *is* a fingerprint

An embedding is a vector of floats `[f₀, f₁, …, f_{D-1}]`. Each component is decomposed exactly as in §4/§7:

```
f_i = mantissa_i / 10^(S_i)          # e.g. 0.123456 = 123456 / 10^6
```

So an embedding's **fingerprint** is two things:

- the vector of integer **mantissas** (each encodable as 5-bit digit tokens), and
- the vector of **scales** `S_i` (the decimal exponents).

No separate "embedding column," no float32 blob to maintain alongside the row. The record's own tokens *are* the vector.

12.2 S selection (the quantization knob)

`S` controls precision, and choosing it is the only real design decision:

- **Uniform S** (one scale for all D components) is the common case for a given model's outputs: quantize every component to `S` decimals. This makes every fingerprint a **fixed-width integer lane vector** — ideal for SIMD/SIMT — and makes the S-bucket trivial (all rows share the bucket).
- **Per-component S** allows mixed precision when some dimensions need more resolution than others.
- Higher S → finer resolution, longer fingerprints, more bits to compare. Lower S → coarser, shorter, faster. It is the native equivalent of fixed-point / binary quantization, with no extension required.

[NOTE] Use **canonical form**: strip trailing zeros (or pin S per field) so that equal values land in the same S-bucket ($0.5 = 5/10^1$, never $50/10^2$), and **pad to equal length within a bucket** so XOR/Hamming compare aligned lanes.

12.3 The three operations

All three are XOR / popcount / sum-of-absolute-differences over fixed-width lanes — the exact primitives a GPU is built to run in parallel:

Query	Mechanism
Exact match	`S` matches (XOR S == 0) **and** `popcount(XOR(mantissa fingerprints)) == 0`
Hamming distance	`popcount(XOR(fingerprints))` — bit-level closeness
	`Σ |mantissa_a − mantissa_b|` at matched S — value-space

Manhattan (L1)	closeness

Exact answers "is this the same vector," Hamming answers "how many bits differ," Manhattan answers "how far apart in value." One substrate, three distance semantics, zero extensions.

12.4 Why this is GPU-native

- The data is **already vectors** — no featurization/serialization step.
- Fixed-width lanes → coalesced memory access; XOR, popcount, and SAD are textbook SIMT kernels.
- It's a **brute-force parallel sweep with no graph to build**. The O(n) work is unchanged but wall-clock collapses to ~O(n / cores); the index-build and index-maintenance costs of HNSW/IVFFlat simply don't exist.
- On-disk/wire it stays 5-bit packed; on load it expands to int8/int16/int32 lanes in VRAM for the kernels (same pattern Arrow uses).

12.5 Verified results [VERIFIED]

CPU, NumPy-vectorized (the CPU mirror of the GPU kernel), 50,000 embeddings × 16 dims, S=6, no index:

```
EXACT   (S-bucket + XOR popcount==0): correct, ~1.26M comparisons/s
HAMMING nearest                     : correct (returns the query row)
MANHATTAN nearest                   : correct, fastest of the three (pure int SAD
```

And on single-scalar exact match, the same mechanism beat Postgres (seq scan) **2x on CPU** for 1,000 floats, both returning the correct row. See `bench_vector_fingerprint.py` and `bench_fingerprint_pg.py`.

12.6 Honest scope [NOTE]

- These are **exact and brute-force** results, correct and GPU-shaped, measured on CPU. The GPU kernel numbers and the >1M-vector scale comparison against `pgvector + HNSW` are the **next experiment**, not yet run. The mechanism's shape (XOR/popcount/SAD) is what justifies expecting the gap to widen on GPU, but that is design reasoning until measured.
- For sub-linear search at very large scale without scanning every row, the intended next layer is **learned / transformer retrieval over the fabric** (attention as the index), with

Hamming-0 as the cheap exact-verify step. That is the frontier, not a shipped feature.

- vs Postgres: exact scalar match is native to both (`=`). The decisive bolt-on gap is **vector** Hamming/Manhattan, which Postgres needs **pgvector** (a compiled extension + a separate vector column) to do, and 5bit does natively with the same two steps at any dimensionality.

---

13. Quick reference

```
Token constants:
  D0..D9      = 0..9
  ops + - * / = ( )  = 10..16        (NUM context)
  N1..N9      = 17..25               (negative digits, NUM context)
  POW (^)     = 26
  SCALE (S)   = 27                   (decimal scale marker, NUM)
  RECORD      = 28   CHECKSUM = 29   END = 30   START = 31

Stack:  NUM ⇄ WORD ⇄ SPECIAL ⇄ SPECIAL2 ⇄ SPECIAL3   (START down, END up)
Pack:   5 bits/token, MSB-first, zero-pad tail; pad = (8 - 5n % 8) % 8
Record: interleaved  LABEL pos name | value | … | RECORD
```